Meet YOLO26: the next generation of real-time vision AI.

Ultralytics YOLOv5 Architecture

YOLOv5 (v6.0/6.1) is a powerful object detection algorithm developed by Ultralytics. This article dives deep into the YOLOv5 architecture, data augmentation strategies, training methodologies, and loss computation techniques. This comprehensive understanding will help improve your practical application of object detection in various fields, including surveillance, autonomous vehicles, and image recognition.

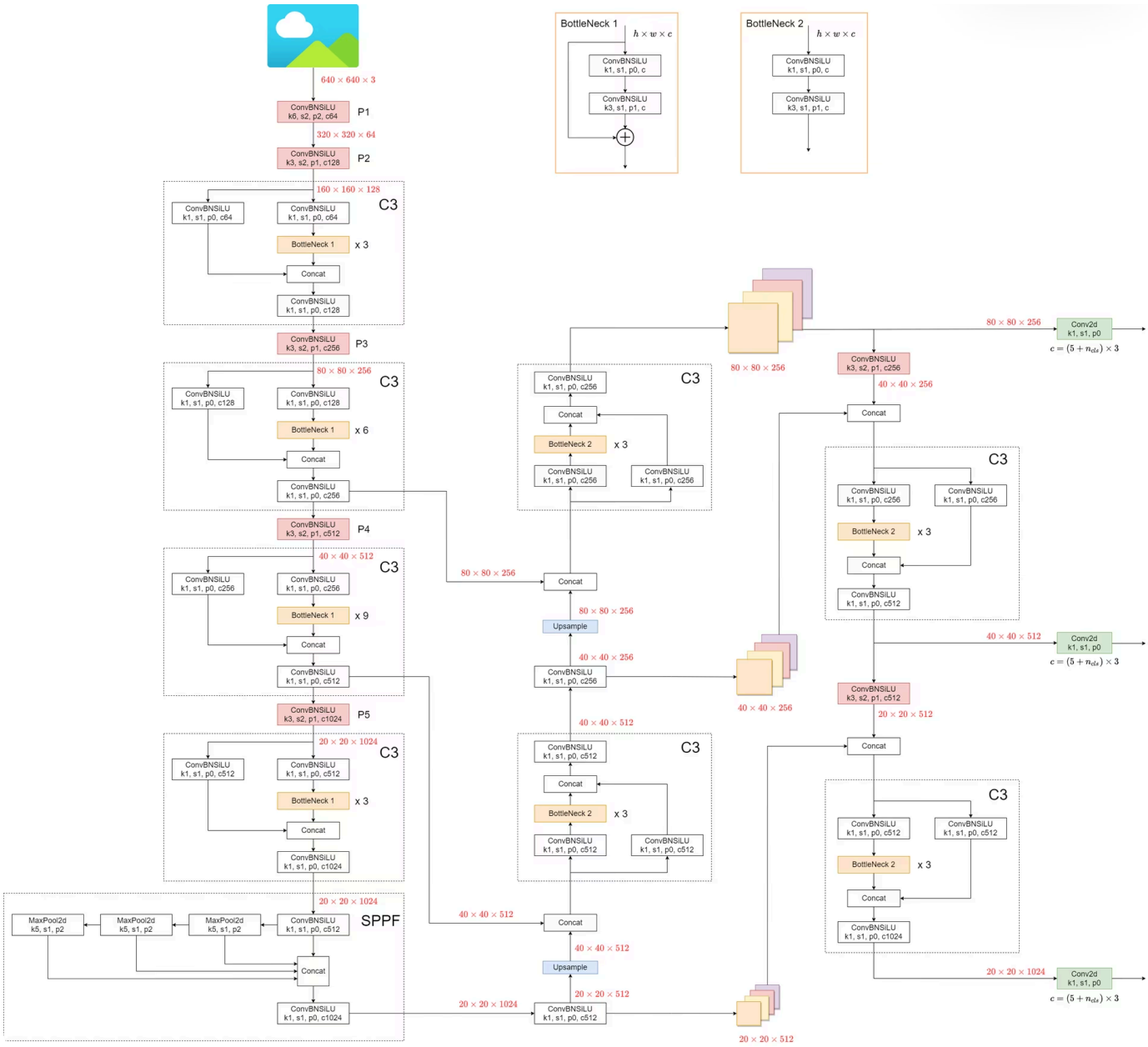
1. Model Structure

YOLOv5's architecture consists of three main parts:

- **Backbone**: This is the main body of the network. For YOLOv5, the backbone is designed using the `CSPDarknet53` structure, a modification of the Darknet architecture used in previous versions.
- **Neck**: This part connects the backbone and the head. In YOLOv5, `SPPF` (Spatial Pyramid Pooling - Fast) and `PANet` (Path Aggregation Network) structures are utilized.
- **Head**: This part is responsible for generating the final output. YOLOv5 uses the `YOLOv3 Head` for this purpose.

The structure of the model is depicted in the image below. The model structure details can be found in `models/yolov5l.yaml`.

Ask AI



YOLOv5 introduces some notable improvements compared to its predecessors:

1. The `Focus` structure, found in earlier versions, is replaced with a `6x6 Conv2d` structure. This change boosts efficiency #4825.
2. The `SPP` structure is replaced with `SPPF`. This alteration more than doubles the speed of processing while maintaining the same output.

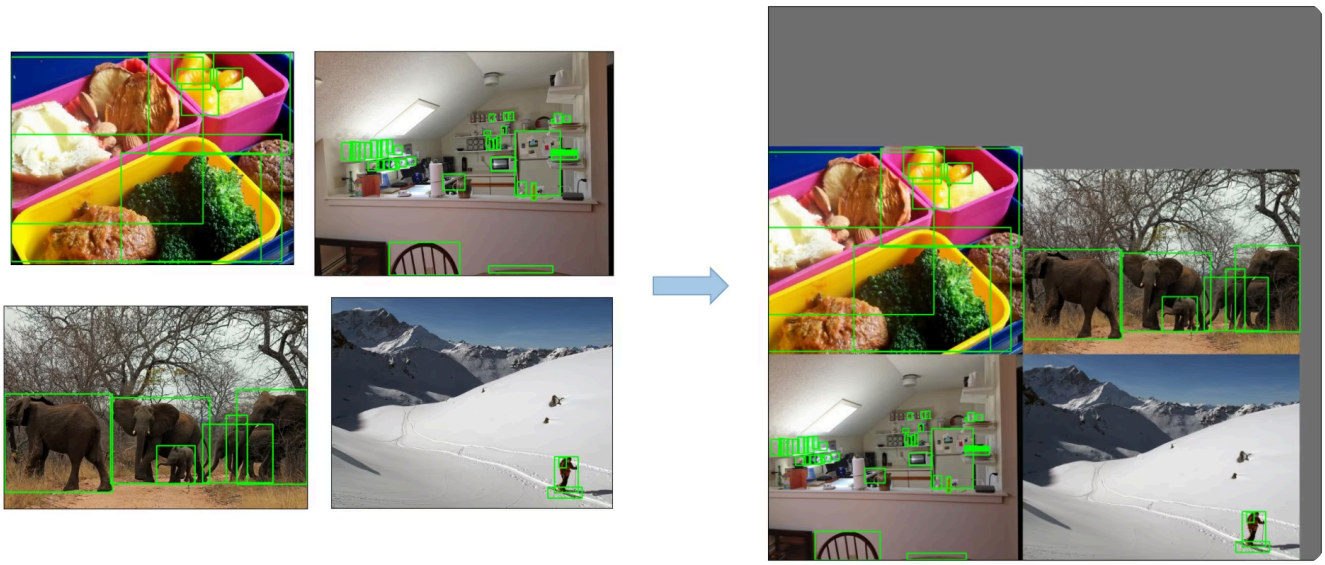
To test the speed of `SPP` and `SPPF`, the following code can be used:

▶ SPP vs SPPF speed profiling example (click to open)

2. Data Augmentation Techniques

YOLOv5 employs various data augmentation techniques to improve the model's ability to generalize and reduce overfitting. These techniques include:

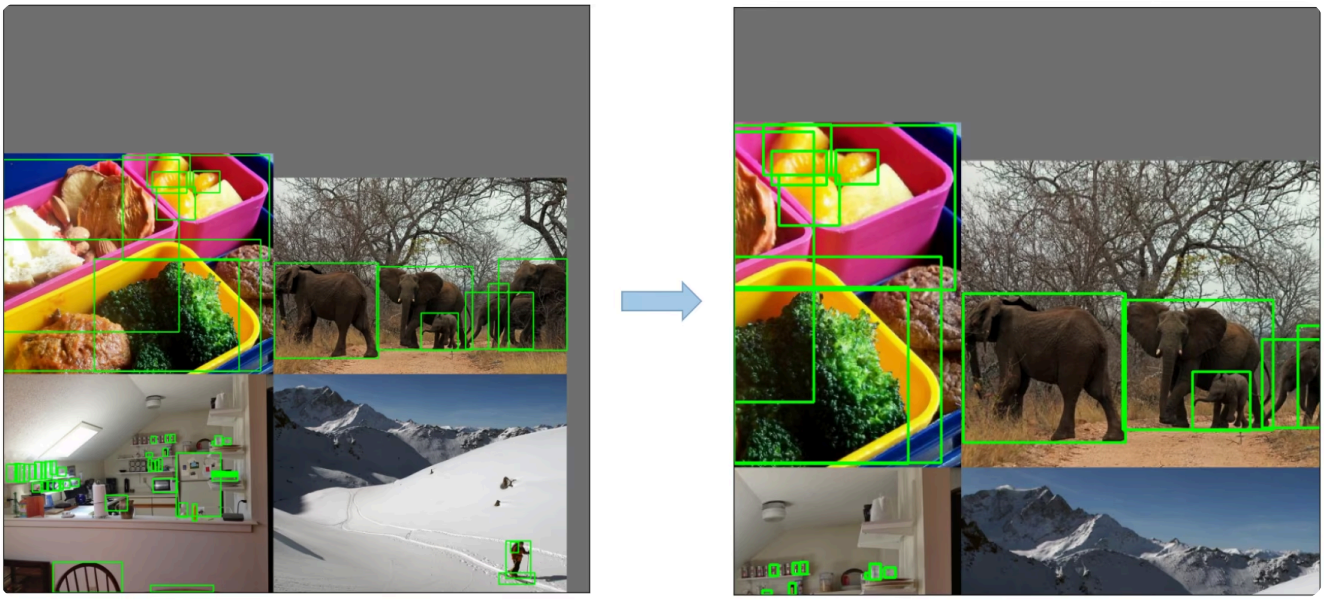
- **Mosaic Augmentation**: An image processing technique that combines four training images into one in ways that encourage object detection models to better handle various object scales and translations.



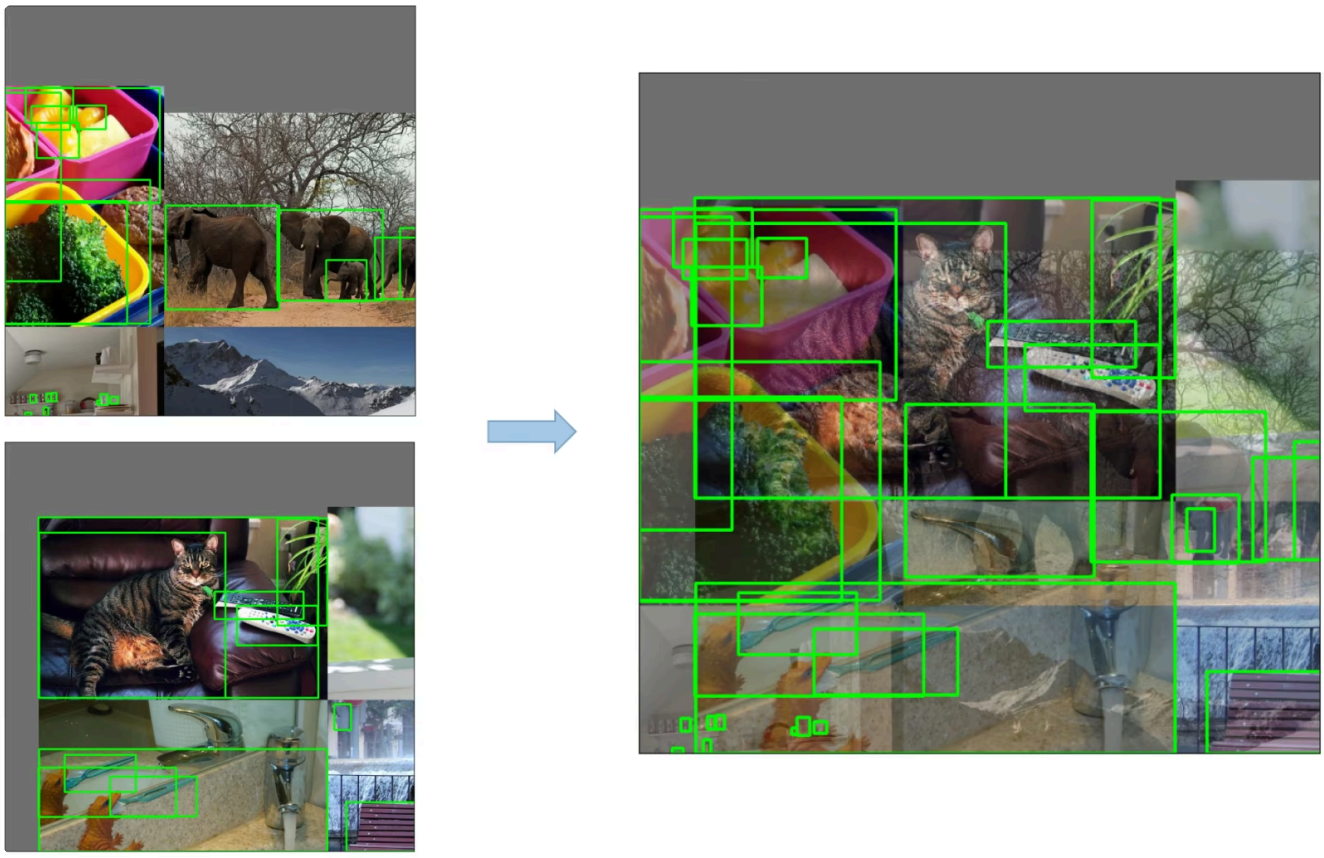
- **Copy-Paste Augmentation**: An innovative data augmentation method that copies random patches from an image and pastes them onto another randomly chosen image, effectively generating a new training sample.



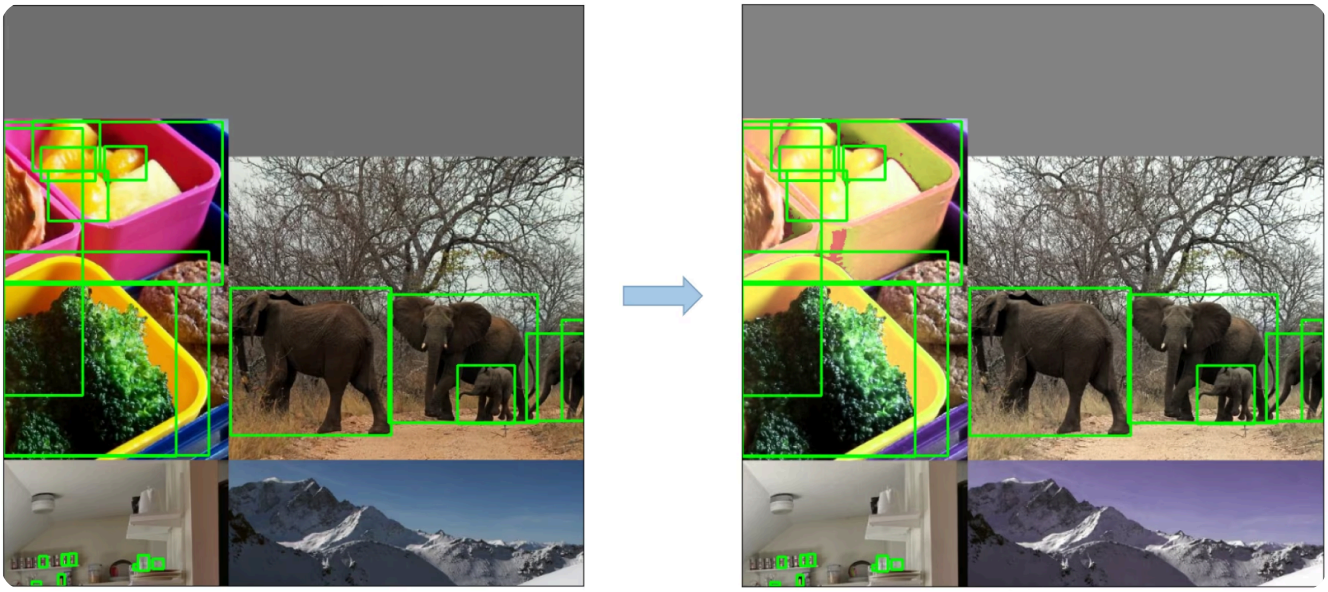
- **Random Affine Transformations**: This includes random rotation, scaling, translation, and shearing of the images.



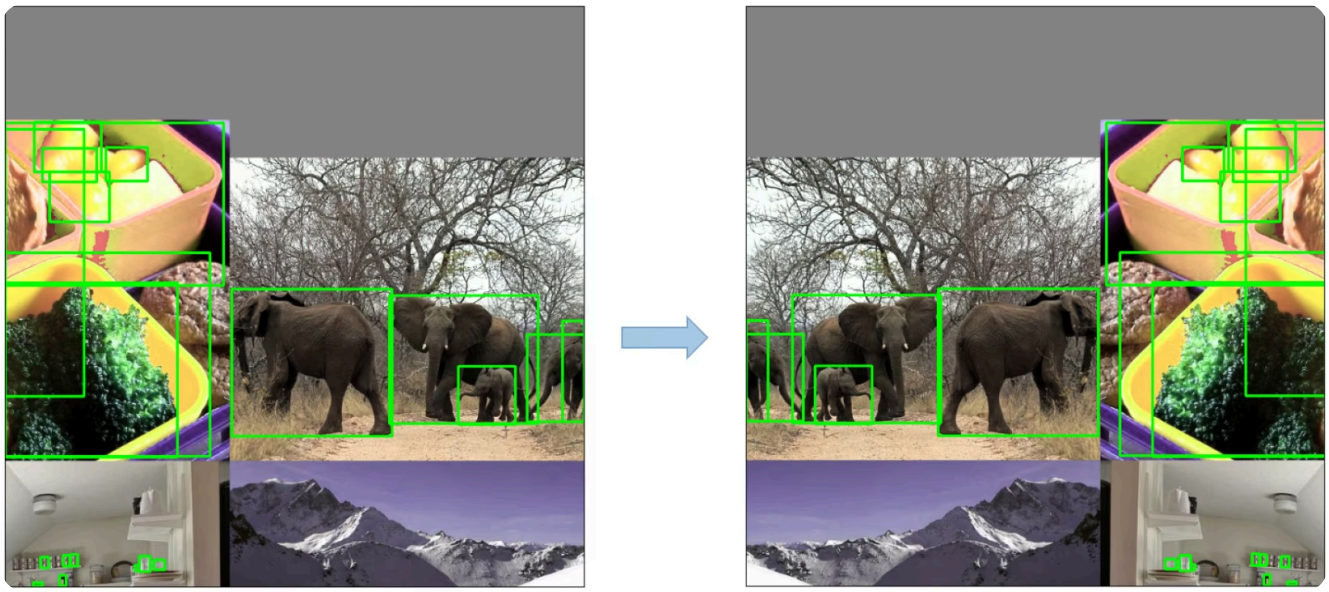
- **MixUp Augmentation**: A method that creates composite images by taking a linear combination of two images and their associated labels.



- **Albumentations**: A powerful image augmentation library that supports a wide variety of augmentation techniques. Learn more about using Albumentations augmentations.
- **HSV Augmentation**: Random changes to the Hue, Saturation, and Value of the images.



- **Random Horizontal Flip**: An augmentation method that randomly flips images horizontally.



3. Training Strategies

YOLOv5 applies several sophisticated training strategies to enhance the model's performance. They include:

- **Multiscale Training**: The input images are randomly rescaled within a range of 0.5 to 1.5 times their original size during the training process.
- **AutoAnchor**: This strategy optimizes the prior anchor boxes to match the statistical characteristics of the ground truth boxes in your custom data.
- **Warmup and Cosine LR Scheduler**: A method to adjust the learning rate to enhance model performance.

- **Exponential Moving Average (EMA)**: A strategy that uses the average of parameters over past steps to stabilize the training process and reduce generalization error.
- **Mixed Precision Training**: A method to perform operations in half-precision format, reducing memory usage and enhancing computational speed.
- **Hyperparameter Evolution**: A strategy to automatically tune hyperparameters to achieve optimal performance. Learn more about hyperparameter tuning.

4. Additional Features

4.1 Compute Losses

The loss in YOLOv5 is computed as a combination of three individual loss components:

- **Classes Loss (BCE Loss)**: Binary Cross-Entropy loss, measures the error for the classification task.
- **Objectness Loss (BCE Loss)**: Another Binary Cross-Entropy loss, calculates the error in detecting whether an object is present in a particular grid cell or not.
- **Location Loss (CIoU Loss)**: Complete IoU loss, measures the error in localizing the object within the grid cell.

The overall loss function is depicted by:

$$Loss = \lambda_1 L_{cls} + \lambda_2 L_{obj} + \lambda_3 L_{loc}$$

4.2 Balance Losses

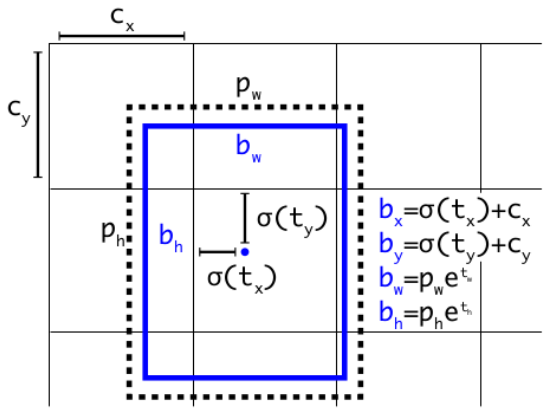
The objectness losses of the three prediction layers (`P3` , `P4` , `P5`) are weighted differently. The balance weights are `[4.0, 1.0, 0.4]` respectively. This approach ensures that the predictions at different scales contribute appropriately to the total loss.

$$L_{obj} = 4.0 \cdot L_{obj}^{small} + 1.0 \cdot L_{obj}^{medium} + 0.4 \cdot L_{obj}^{large}$$

4.3 Eliminate Grid Sensitivity

The YOLOv5 architecture makes some important changes to the box prediction strategy compared to earlier versions of YOLO. In YOLOv2 and YOLOv3, the box coordinates were directly predicted using the activation of the last layer.

$$
\begin{aligned}
b_x &= \sigma(t_x) + c_x \\
b_y &= \sigma(t_y) + c_y \\
b_w &= p_w \cdot e^{t_w} \\
b_h &= p_h \cdot e^{t_h}
\end{aligned}
$$

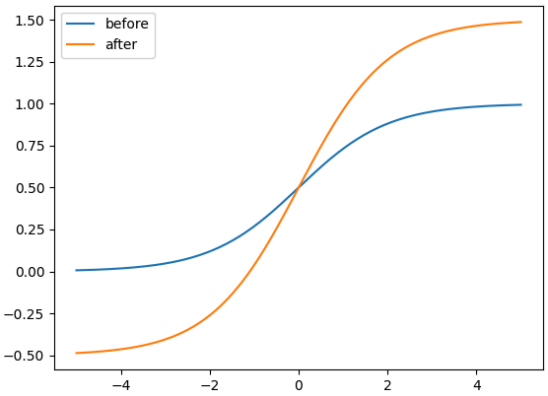


However, in YOLOv5, the formula for predicting the box coordinates has been updated to reduce grid sensitivity and prevent the model from predicting unbounded box dimensions.

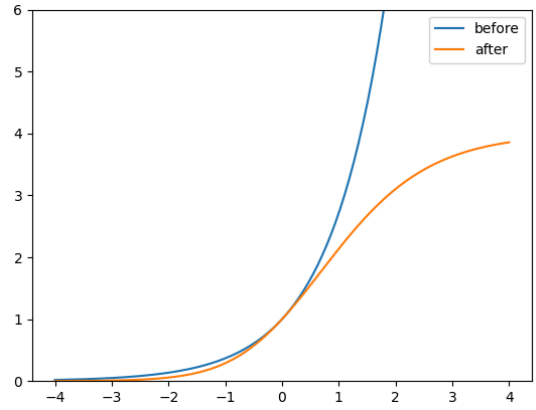
The revised formulas for calculating the predicted bounding box are as follows:

$$
\begin{aligned}
b_x &= (2 \cdot \sigma(t_x) - 0.5) + c_x \\
b_y &= (2 \cdot \sigma(t_y) - 0.5) + c_y \\
b_w &= p_w \cdot (2 \cdot \sigma(t_w))^2 \\
b_h &= p_h \cdot (2 \cdot \sigma(t_h))^2
\end{aligned}
$$

Compare the center point offset before and after scaling. The center point offset range is adjusted from (0, 1) to (-0.5, 1.5). Therefore, offset can easily get 0 or 1.



Compare the height and width scaling ratio (relative to anchor) before and after adjustment. The original yolo/darknet box equations have a serious flaw. Width and Height are completely unbounded as they are simply out=exp(in), which is dangerous, as it can lead to runaway gradients, instabilities, NaN losses and ultimately a complete loss of training. Refer to this issue for more details.



4.4 Build Targets

The build target process in YOLOv5 is critical for training efficiency and model accuracy. It involves assigning ground truth boxes to the appropriate grid cells in the output map and matching them with the appropriate anchor boxes.

This process follows these steps:

- Calculate the ratio of the ground truth box dimensions and the dimensions of each anchor template.

$$r_w = w_{gt} / w_{at}$$

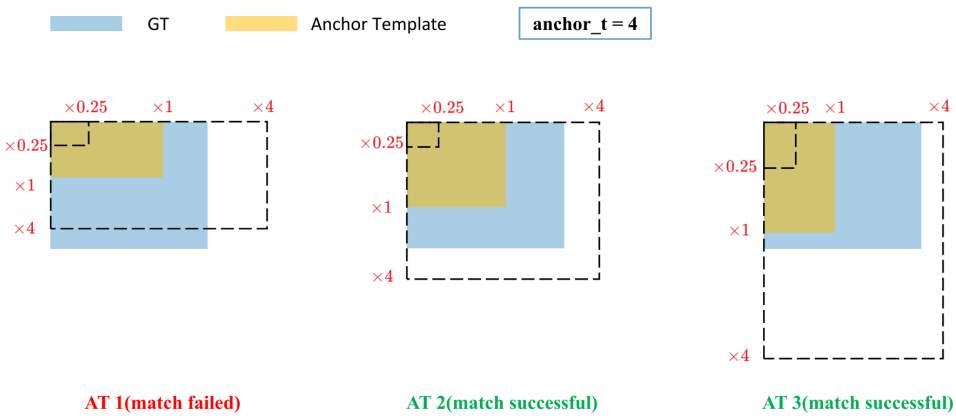
$$r_h = h_{gt} / h_{at}$$

$$r_w^{max} = max(r_w, 1/r_w)$$

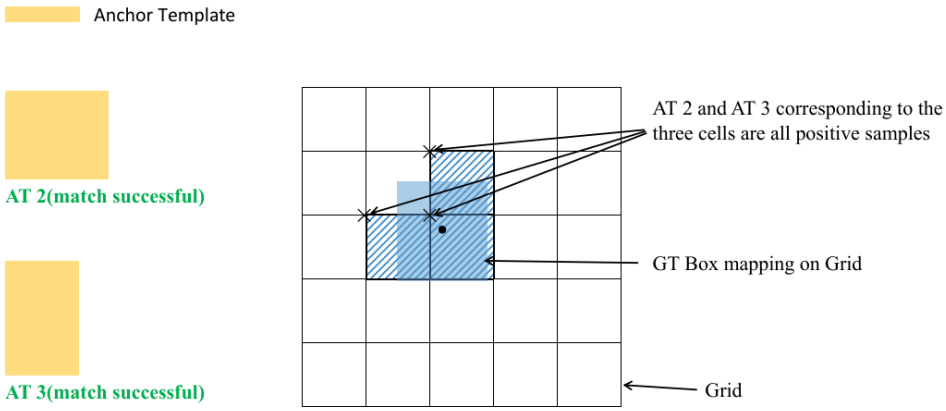
$$r_h^{max} = max(r_h, 1/r_h)$$

$$r^{max} = max(r_w^{max}, r_h^{max})$$

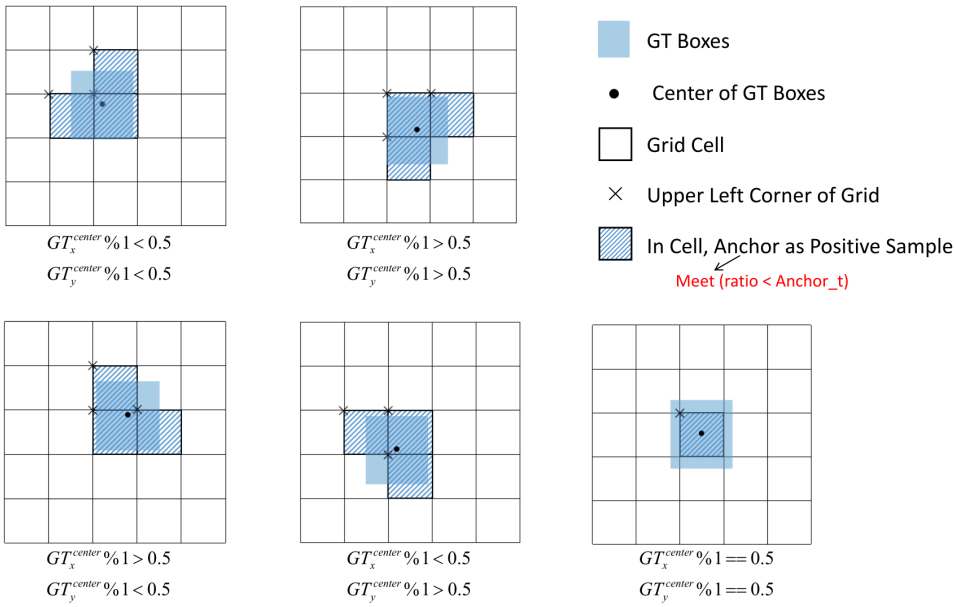
$$r^{max} < \text{anchor}_t$$



- If the calculated ratio is within the threshold, match the ground truth box with the corresponding anchor.



- Assign the matched anchor to the appropriate cells, keeping in mind that due to the revised center point offset, a ground truth box can be assigned to more than one anchor because the center point offset range is adjusted from (0, 1) to (-0.5, 1.5), making additional matches possible.



This way, the build targets process ensures that each ground truth object is properly assigned and matched during the training process, allowing YOLOv5 to learn the task of object detection more effectively.

Conclusion

YOLOv5 represents a meaningful step in the evolution of real-time object detection. Its architectural choices, training strategies, and engineering refinements deliver strong performance and efficiency relative to earlier YOLO versions.

The primary enhancements in YOLOv5 include the use of a dynamic architecture, an extensive range of data augmentation techniques, innovative training strategies, as well as

important adjustments in computing losses and the process of building targets. All these innovations significantly improve the accuracy and efficiency of object detection while retaining a high degree of speed, which is the trademark of YOLO models.

Contributors

 glenn-jocher [16]

 RizwanMunawar [1]

 ambitious-octopus [1]

Created Nov 12, 2023 Updated 2 weeks ago

Comments

PREVIOUS
Transfer Learning with Frozen Layers in ...

NEXT
Deploy YOLOv5 with Neural Magic's Dee...